

```

# --- Setup (use a TF version Colab supports)
# If Colab already has TF >= 2.18, you can skip the next line.
!pip install -q "tensorflow>=2.18,<2.21" scikit-learn

import os, numpy as np, random, tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
print("TensorFlow version:", tf.__version__)
tf.random.set_seed(42); np.random.seed(42); random.seed(42)

# --- Load IMDB (pre-tokenized integer sequences)
(train_data, train_labels), (test_data, test_labels) = keras.datasets.imdb.load_data(num_words=20000)

# --- Pad sequences
MAXLEN = 200
x_train = keras.preprocessing.sequence.pad_sequences(train_data, maxlen=MAXLEN)
x_test = keras.preprocessing.sequence.pad_sequences(test_data, maxlen=MAXLEN)
y_train, y_test = np.array(train_labels), np.array(test_labels)

# --- Build LSTM model
EMB_DIM = 128
model = keras.Sequential([
    layers.Embedding(input_dim=20000, output_dim=EMB_DIM, input_length=MAXLEN),
    layers.SpatialDropout1D(0.2),
    layers.Bidirectional(layers.LSTM(128, return_sequences=True)),
    layers.GlobalMaxPooling1D(),
    layers.Dropout(0.4),
    layers.Dense(64, activation="relu"),
    layers.Dropout(0.3),
    layers.Dense(1, activation="sigmoid")
])
model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss="binary_crossentropy", metrics=["accuracy"])
model.summary()

# --- Train
callbacks = [
    keras.callbacks.EarlyStopping(patience=3, restore_best_weights=True),
    keras.callbacks.ReduceLROnPlateau(patience=2, factor=0.5),
]
history = model.fit(x_train, y_train, validation_split=0.1,
                    epochs=10, batch_size=128, callbacks=callbacks, verbose=2)

# --- Evaluate & report
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)
print(f"Test accuracy: {test_acc:.4f}")

from sklearn.metrics import classification_report
y_pred = (model.predict(x_test, verbose=0).ravel() > 0.5).astype("int32")
print(classification_report(y_test, y_pred, target_names=["neg", "pos"]))

# --- Save artifacts (Keras 3 requires an extension)
model.save("lstm_sentiment_model.keras") # native Keras format
# If you need HDF5 instead: model.save("lstm_sentiment_model.h5")
np.savez("lstm_metrics_imdb.npz", test_acc=test_acc)

```

TensorFlow version: 2.19.0
 Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz>
 17464789/17464789 — 0s 0us/step
 /usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97: UserWarning: Argument `input\_length` is deprecated
 warnings.warn(
 Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
spatial_dropout1d (SpatialDropout1D)	?	0
bidirectional (Bidirectional)	?	0 (unbuilt)
global_max_pooling1d (GlobalMaxPooling1D)	?	0
dropout (Dropout)	?	0
dense (Dense)	?	0 (unbuilt)
dropout_1 (Dropout)	?	0
dense_1 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)
 Trainable params: 0 (0.00 B)
 Non-trainable params: 0 (0.00 B)

Epoch 1/10
 176/176 - 15s - 84ms/step - accuracy: 0.6964 - loss: 0.5636 - val_accuracy: 0.8568 - val_loss: 0.3455 - learning_rate: 1.0000e-0
 Epoch 2/10
 176/176 - 6s - 36ms/step - accuracy: 0.8793 - loss: 0.2949 - val_accuracy: 0.8688 - val_loss: 0.3186 - learning_rate: 1.0000e-03
 Epoch 3/10
 176/176 - 6s - 36ms/step - accuracy: 0.9319 - loss: 0.1845 - val_accuracy: 0.8596 - val_loss: 0.4583 - learning_rate: 1.0000e-03
 Epoch 4/10
 176/176 - 6s - 37ms/step - accuracy: 0.9501 - loss: 0.1371 - val_accuracy: 0.8600 - val_loss: 0.4808 - learning_rate: 1.0000e-03
 Epoch 5/10
 176/176 - 6s - 37ms/step - accuracy: 0.9752 - loss: 0.0754 - val_accuracy: 0.8732 - val_loss: 0.4237 - learning_rate: 5.0000e-04
 Test accuracy: 0.8561

	precision	recall	f1-score	support
neg	0.82	0.91	0.86	12500
pos	0.90	0.80	0.85	12500
accuracy			0.86	25000
macro avg	0.86	0.86	0.86	25000
weighted avg	0.86	0.86	0.86	25000

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.